Simplicial Complex Multiset Embedders

Algorithms 1 and 2 — Strategy and Implementation (v2)

June 12, 2026

Contents

1 What We Are Trying to Do

The input is an unordered multiset $M = \{\mathbf{v}_0, \ldots, \mathbf{v}_{n-1}\}$ of n vectors in $\mathbb{R}^k$. Concretely we represent M as a $k \times n$ matrix whose columns are the vectors (column j = vector $\mathbf{v}_j$, row i = coordinate i). Column order is arbitrary.

We want a map f from such multisets to some $\mathbb{R}^N$ that is:

- **Multiset-invariant** — the output does not depend on how the columns are ordered.
- **Injective** — distinct multisets give distinct outputs (f is an *embedder*).
- **Continuous** (C^0) — small changes in the vectors produce small changes in $f(M)$.
- **Positively homogeneous of degree 1** — $f(\lambda M) = \lambda f(M)$ for all $\lambda > 0$.

Multiset invariance and continuity are in tension: any map that achieves invariance by sorting will have kinks wherever the sorted order changes. The strategy below resolves this tension elegantly.

2 The Strategy

Step A — Write M in a natural basis

Label each of the nk matrix entries by e_i^j (column j, row i). Let $X[j][i]$ denote the entry in column j, row i. Then

$$M = \sum_{j,i} X[j][i]\, e_i^j,$$

where e_i^j is the $k \times n$ matrix that is 1 at (i, j) and 0 elsewhere. This is the expansion of M in the *standard Eji basis*.

Step B — Peel off the fully symmetric parts

A *fully symmetric* basis element at row i is $\sum_j e_i^j$: it places the same coefficient on every vector at coordinate i, so it is unaffected by any permutation of the columns. The smallest coefficient that any vector has at row i is $m_i = \min_j X[j][i]$. Factoring:

$$M \;=\; \underbrace{\sum_i m_i \left(\sum_j e_i^j \right)}_{\text{symmetric part}} \;+\; \underbrace{\text{balance}}_{\text{all entries} \geq 0} \;.$$

The symmetric part is already permutation-invariant; its k scalar coefficients $m_0, \ldots, m_{k-1}$ can be stored directly as anchor values.

The balance $M - \sum_i m_i (\sum_j e_i^j)$ has all non-negative entries and contains the non-trivial, non-symmetric information in M.

Step C — Re-express the balance as a positive combination of nested basis elements

Within row i, order the n vectors descending by their (shifted) value. Let $\sigma_i(0), \ldots, \sigma_i(n-1)$ be this ordering. The gaps $\delta_r^{(i)} = X[\sigma_i(r)][i] - X[\sigma_i(r+1)][i] \geq 0$ are the drops between consecutive ranks.

By Abel summation (the discrete analogue of integration by parts), the balance at row i becomes:

$$\text{balance at row } i \;=\; \sum_{r=0}^{n-2} \delta_r^{(i)} \cdot V_r^{(i)},$$

where the *prefix basis element* $V_r^{(i)} = \{ e_i^{\sigma_i(0)}, \ldots, e_i^{\sigma_i(r)} \}$ is the set of the top-$(r{+}1)$ vectors by row-i value.

This is another change of basis: from individual Ejis with signed coefficients to nested prefix sets with non-negative gap coefficients. The structure of which vectors appear in $V_r^{(i)}$ — which j's dominate row i — is the non-symmetric information not yet discarded.

Step D — Merge, re-sort, and apply a second Abel summation

Collect all $m = k(n-1)$ gap–prefix-element pairs from all rows into one list and sort descending by gap: $\delta_{(0)} \geq \cdots \geq \delta_{(m-1)}$ with corresponding basis elements $V_{(0)}, \ldots, V_{(m-1)}$.

Apply a second Abel summation to obtain *cumulative* basis elements and weighted-difference coefficients:

$$\alpha_s \;=\; (s+1)\big(\delta_{(s)} - \delta_{(s+1)}\big), \qquad \delta_{(m)} := 0,$$

$$L_s \;=\; V_{(0)} + V_{(1)} + \cdots + V_{(s)},$$

where L_s is the *Eji_LinComb*: the $k \times n$ matrix whose (i, j) entry counts how many of the first $s+1$ prefix elements contain e_i^j. Its *index* $s+1$ records how many prefix elements have been accumulated.

The mathematical identity $\sum_s \delta_{(s)} V_{(s)} = \sum_s \alpha_s \cdot (L_s/(s+1))$ confirms this preserves all information. The normalised basis element $L_s/(s+1)$ is the *average* of the first $s+1$ prefix elements. The $(s+1)$ weight in α_s distributes contributions more evenly and ensures that $\sum_s \alpha_s = \sum_s \delta_{(s)}$ (the total weight is preserved).

Crucially, L_s captures *which prefix elements came first* in the global sort: two different orderings of the same set of prefix elements produce different L_s sequences.

Step E — Achieve permutation invariance by canonicalising

The j-indices in L_s still reflect the arbitrary labelling of columns. The *canonical form* $\widetilde{L_s}$ is obtained by sorting the columns of L_s's count matrix lexicographically — this removes the column labelling while retaining the multiset structure. Two inputs related by a column permutation produce identical $\widetilde{L_s}$ sequences and hence identical outputs.

Step F — Encode in $\mathbb{R}^{2m+1}$ and assemble

Map each canonical L_s to a pseudorandom point $p_s = h(\widetilde{L_s}) \in [0,1]^{2m+1}$ via MD5 hashing of its count matrix and index. The main body of the output is $\sum_s \alpha_s p_s$.

Why $\mathbb{R}^{2m+1}$ is the right dimension. All $\alpha_s \geq 0$, so $\sum_s \alpha_s p_s$ is a point in the *positive cone* generated by $\{p_0, \ldots, p_{m-1}\}$, a cone of dimension m. Two m-dimensional positive cones in $\mathbb{R}^{2m+1}$ are *generically disjoint* by dimension counting: $m + m = 2m < 2m + 1$. Therefore a single point in $\mathbb{R}^{2m+1}$ encodes both: (a) which cone it lies in (i.e. which canonical $\{L_s\}$, i.e. which basis elements), and (b) the coefficients $\{\alpha_s\}$ within that cone.
This is also why the algorithm is C^0: when two values in the same row are equal, the gap at that position is 0 and so $\alpha_s = 0$. The point $\sum \alpha_s p_s$ then lies on a shared *face* of two adjacent cones rather than in either interior. That shared face is the geometric expression of the continuity: the change in which cone we are in happens exactly at the moment when the coefficient of the transitioning basis element is zero, so the output is unaffected.

The full output is:

$$f(M) \;=\; \Big[\, \underbrace{m_0, \ldots, m_{k-1}}_{k \text{ row minima}},\; \underbrace{\textstyle\sum_{s=0}^{m-1} \alpha_s p_s}_{2m+1 \text{ reals}} \,\Big] \;\in\; \mathbb{R}^{k+2m+1}.$$

With $m = k(n-1)$: total size $= k + 2k(n-1) + 1 = 2nk + 1 - k$.

3 Two Algorithms, One Strategy

Both algorithms follow Steps A–F. They differ only in **Step B**: how many symmetric parts are peeled off, and what "fully symmetric" means.

Algorithm 2 (per-row). Factors out the k per-row symmetric elements $\sum_j e_i^j$, one per coordinate. Stores k row minima as anchors. Steps C–F operate on $m = k(n-1)$ gaps. Implemented as `Embedder.embed_generic` in `COHomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py` (direct) and `COHomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py` (EncDec-backed); the `COHomDeg1_simplicialComplex_embedder_2_for_array_of_reals_as_multiset.py` wrapper selects between them. See also `simplex_2_preprocess_steps` in `EncDec.py` and Appendix A.

Algorithm 1 (global). Factors out only the *single* fully-symmetric element $\sum_{i,j} e_i^j$ (treating every entry identically), with coefficient = global minimum. The global maximum is also recorded (see note below). Steps C–F then operate on a single globally-sorted list of $m = nk - 1$ gaps. Implemented as `Embedder.embed_generic` in `COHomDeg1_simplicialComplex_embedder_1_for_array_of_reals_as_multiset.py` and as `simplex_1_preprocess_steps` in `EncDec.py`; see Appendix A.

The single global sort of Algorithm 1 means that prefix elements can contain Ejis from *any* row, whereas in Algorithm 2 every prefix element contains Ejis from exactly one row. This is why Algorithm 2 is conceptually cleaner: it respects the coordinate structure of the vectors.

Dimension count.

	Algorithm 1	Algorithm 2
Symmetric parts extracted	1 (global)	k (per-row)
Gaps m	$nk-1$	$k(n-1) = nk-k$
Anchor values stored	2 (max & min)	k (row minima)
Hash embedding dimension $2m+1$	$2nk-1$	$2nk-2k+1$
Total output size	$\mathbf{2nk+1}$	$\mathbf{2nk+1-k}$

Algorithm 2 saves k output dimensions because per-row symmetric extraction produces $k-1$ fewer gaps than global extraction (since each row contributes $n-1$ gaps, totalling $k(n-1) = nk-k$, versus the global $nk-1$ gaps from Algorithm 1), and the $(s+1)$ weighting scheme in Step D ensures that the hash dimension savings exceed the extra anchor overhead.

Note on Algorithm 1's global maximum. In a perfect implementation, only the global minimum is needed as an anchor (it fixes the absolute scale; all relative gaps are encoded in the hash sum). The global maximum is technically redundant. It is included in Algorithm 1 for implementation simplicity; a TODO comment in the source code acknowledges this.

Source code.

Algorithm 2 — two interchangeable implementations (Steps A–F each):

- `COHomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py` — direct implementation; `Embedder.embed_generic`. This is the original file on which the document is based.
- `COHomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py` — EncDec-backed; `Embedder.embed_generic`. Delegates Steps A–E to `EncDec.py` and adds Step F directly. Default config produces bit-identical output to `COHomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py`.
- `COHomDeg1_simplicialComplex_embedder_2_for_array_of_reals_as_multiset.py` — wrapper that selects between the above. All other code imports `Embedder` from this file.

Algorithm 1:

- `COHomDeg1_simplicialComplex_embedder_1_for_array_of_reals_as_multiset.py` — direct implementation; `Embedder.embed_generic`; Steps A–F.

Shared infrastructure:

- `EncDec.py` — `simplex_2_preprocess_steps` and `simplex_1_preprocess_steps` cover Steps A–E. Docstrings use this document's $\delta_{(s)}$, $V_{(s)}$, $\Delta_{(s)}$, $L_{(s)}$, α_s, $\hat{L}_{(s)}$ notation.
- `play_simplex_encoders_via_EncDec.py` — command-line driver for `EncDec.py`.

A parity test suite (`test_simplex2_ab_parity.py`) verifies that `COHomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py` and `COHomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py` produce identical embeddings under the default configuration. See Appendix A for a full notation–code correspondence table.

4 Worked Example: Algorithm 2, $n = 4$, $k = 3$

$$M = \left\{ \begin{pmatrix} 4 \\ 2 \\ 3 \end{pmatrix}, \begin{pmatrix} -3 \\ 5 \\ 1 \end{pmatrix}, \begin{pmatrix} 8 \\ 9 \\ 2 \end{pmatrix}, \begin{pmatrix} 2 \\ 7 \\ 2 \end{pmatrix} \right\} \quad (n = 4 \text{ vectors in } \mathbb{R}^3)$$

Step B: Extract row minima

Row minima: $m_0 = -3$ (row 0), $m_1 = 2$ (row 1), $m_2 = 1$ (row 2). These three scalars are stored as anchors. After subtracting:

$$\text{balance} = \left\{ \begin{pmatrix} 7 \\ 0 \\ 2 \end{pmatrix}, \begin{pmatrix} 0 \\ 3 \\ 0 \end{pmatrix}, \begin{pmatrix} 11 \\ 7 \\ 1 \end{pmatrix}, \begin{pmatrix} 5 \\ 5 \\ 1 \end{pmatrix} \right\}$$

(All non-negative.) The debug check in the source code verifies that $\sum_s \alpha_s \cdot (L_s/(s+1)) +$ broadcast(m_i) recovers the original M exactly.

Step C: Per-row Abel summation

For each row, sort vectors descending and compute gaps:

Row i	Sorted descent (value: vector j)	Gaps δ	Prefix elements $V_r^{(i)}$
0	11(j=2), 7(j=0), 5(j=3), 0(j=1)	4, 2, 5	$\{e_0^2\}$; $\{e_0^2, e_0^0\}$; $\{e_0^2, e_0^0, e_0^3\}$
1	7(j=2), 5(j=3), 3(j=1), 0(j=0)	2, 2, 3	$\{e_1^2\}$; $\{e_1^2, e_1^3\}$; $\{e_1^2, e_1^3, e_1^1\}$
2	2(j=0), 1(j=2), 1(j=3), 0(j=1)	1, 0, 1	$\{e_2^0\}$; $\{e_2^0, e_2^2\}$; $\{e_2^0, e_2^2, e_2^3\}$

At this stage in the code, each pair is stored as (δ, `MSV`) where the MSV is an individual prefix set — it is a valid basis element but not yet in its final cumulative form.

Steps D: Merge, sort, and second Abel summation

Merge all $9 = k(n-1)$ pairs and sort by gap (i.e. δ) descending:

$$5,\ 4,\ 3,\ 2,\ 2,\ 2,\ 1,\ 1,\ 0$$

The table below shows to which basis element $V_{(s)}$ each sorted gap $\delta_{(s)}$ was associated, and accumulates these $V_{(s)}$ into $L_s = V_{(0)} + \cdots + V_{(s)}$. Terms within each L_s expression are grouped by coordinate row i.

			$\delta_{(s)} - \delta_{(s+1)}$	$L_{(s-1)} + V_{(s)}$	$(s+1)\Delta_{(s)}$	$L_{(s)}/(s+1)$
s	$\delta_{(s)}$	$V_{(s)}$	$\Delta_{(s)}$	$L_{(s)}$	α_s	$\hat{L}_{(s)}$
0	5	$e_0^0 + e_0^2 + e_0^3$	1	$e_0^0 + e_0^2 + e_0^3$	1	$e_0^0 + e_0^2 + e_0^3$
1	4	e_0^2	1	$e_0^0 + 2e_0^2 + e_0^3$	2	$\frac{1}{2}(e_0^0 + 2e_0^2 + e_0^3)$
2	3	$e_1^1 + e_1^2 + e_1^3$	1	$e_0^0 + 2e_0^2 + e_0^3 + e_1^1 + e_1^2 + e_1^3$	3	$\frac{1}{3}(e_0^0 + 2e_0^2 + e_0^3 + e_1^1 + e_1^2 + e_1^3)$
3	2	$e_0^0 + e_0^2$	0	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + e_1^2 + e_1^3$	0	$\frac{1}{4}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + e_1^2 + e_1^3)$
4	2	e_1^2	0	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 2e_1^2 + e_1^3$	0	$\frac{1}{5}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 2e_1^2 + e_1^3)$
5	2	$e_1^2 + e_1^3$	1	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3$	6	$\frac{1}{6}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3)$
6	1	e_2^0	0	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + e_2^0$	0	$\frac{1}{7}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + e_2^0)$
7	1	$e_2^0 + e_2^2 + e_2^3$	1	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 2e_2^0 + e_2^2 + e_2^3$	8	$\frac{1}{8}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 2e_2^0 + e_2^2 + e_2^3)$
8	0	$e_2^0 + e_2^2$	0	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 3e_2^0 + 2e_2^2 + e_2^3$	0	$\frac{1}{9}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 3e_2^0 + 2e_2^2 + e_2^3)$
	$\sum_s \delta_{(s)} = 20$		$\sum_s \Delta_{(s)} = 5 \neq 20$		$\sum_s \alpha_{(s)} = 20$	
	$\sum_s \delta_{(s)} V_{(s)} = 7e_0^0 + 11e_0^2 + 5e_0^3 + 3e_1^1 + 7e_1^2 + 5e_1^3 + 2e_2^0 + e_2^2 + e_2^3$		$\sum_s \Delta_{(s)} L_{(s)} = 7e_0^0 + 11e_0^2 + 5e_0^3 + 3e_1^1 + 7e_1^2 + 5e_1^3 + 2e_2^0 + e_2^2 + e_2^3$		$\sum_s \alpha_{(s)} \hat{L}_{(s)} = 7e_0^0 + 11e_0^2 + 5e_0^3 + 3e_1^1 + 7e_1^2 + 5e_1^3 + 2e_2^0 + e_2^2 + e_2^3$	

Coincidence in this example. Every non-zero Δ (i.e. every difference between successive δ values) happens to equal 1 here (e.g. $\delta_{(0)} - \delta_{(1)} = 5 - 4 = 1$, $\delta_{(5)} - \delta_{(6)} = 2 - 1 = 1$, etc.), so the non-zero α_s values equal $s{+}1$. This is a coincidence of the chosen data, not a general rule: $\alpha_s = (s{+}1) \times$ (gap difference), and the gap difference can be any non-negative integer.

Verification: column sums and three-way equality. The integer columns $\delta_{(s)}$ and α_s have the same total:

$$\sum_{s=0}^{8}\delta_{(s)} \;=\; 5{+}4{+}3{+}2{+}2{+}2{+}1{+}1{+}0 \;=\; 20 \;=\; 1{+}2{+}3{+}0{+}0{+}6{+}0{+}8{+}0 \;=\; \sum_{s=0}^{8}\alpha_s.$$

The three weighted Eji sums are all equal and recover the `balance` matrix of Step B (the $L_{(s)}$ and $\hat{L}_{(s)}$ totals-row entries show the value directly; $\sum_s \delta_{(s)}V_{(s)}$ is identical):

$$\begin{aligned}\sum_s \delta_{(s)}\,V_{(s)} \;=\; \sum_s \Delta_{(s)}\,L_{(s)} \;=\; \sum_s \alpha_s\,\hat{L}_{(s)} = {}& 7e_0^0 + 11e_0^2 + 5e_0^3 \\ & + 3e_1^1 + 7e_1^2 + 5e_1^3 \\ & + 2e_2^0 + e_2^2 + e_2^3,\end{aligned}$$

i.e. coefficient matrix $\left(\begin{smallmatrix}7&0&11&5\\0&3&7&5\\2&0&1&1\end{smallmatrix}\right)$ (rows $= i = 0,1,2$; columns $= j = 0,1,2,3$), matching `balance` exactly.

In many implementations the basis elements $L_{(s)}$ or $\hat{L}_{(s)}$ are stored as matrices. For example, Eji_LinComb structures https://github.com/kesterlester/Echinocoder/blob/main/Eji_LinComb.py like those shown below (only for each term without a zero coefficient) are used in https://github.com/kesterlester/Echinocoder/blob/main/C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py. In these the count matrix has rows representing coordinates i=0, 1, 2; and columns representing vectors j=0, 1, 2, 3.

s	α_s	Eji_LinComb structure	$\hat{L}_{(s)} = L_{(s)}/(s{+}1)$
0	1	$\begin{pmatrix}1&0&1&1\\0&0&0&0\\0&0&0&0\end{pmatrix}$, idx 1	$e_0^0 + e_0^2 + e_0^3$
1	2	$\begin{pmatrix}1&0&2&1\\0&0&0&0\\0&0&0&0\end{pmatrix}$, idx 2	$\frac{1}{2}(e_0^0 + 2e_0^2 + e_0^3)$
2	3	$\begin{pmatrix}1&0&2&1\\0&1&1&1\\0&0&0&0\end{pmatrix}$, idx 3	$\frac{1}{3}(e_0^0 + 2e_0^2 + e_0^3 + e_1^1 + e_1^2 + e_1^3)$
5	6	$\begin{pmatrix}2&0&3&1\\0&1&3&2\\0&0&0&0\end{pmatrix}$, idx 6	$\frac{1}{6}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3)$
7	8	$\begin{pmatrix}2&0&3&1\\0&1&3&2\\2&0&1&1\end{pmatrix}$, idx 8	$\frac{1}{8}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 2e_2^0 + e_2^2 + e_2^3)$

(Terms with $\alpha_s = 0$ contribute nothing and are omitted.)
The table below repeats the construction with a different but equally valid tie-breaking order: within the three-way $\delta{=}2$ group the assignment is cycled as $V_{(3)} \leftarrow V_{(5)}$, $V_{(4)} \leftarrow V_{(3)}$, $V_{(5)} \leftarrow V_{(4)}$, and within the two-way $\delta = 1$ group the two elements are swapped. The basis vectors in the greyed cells change; the non-greyed cells are identical to those in the table above. Critically the greyed cells all have basis-coefficients (scalars $\Delta_{(s)}$ or α_s) which are zero by construction ... and so the very places where the basis-vectors are ambiguous are also (by construction) the very places where those basis vectors do not need to be encoded. Equivalently the non-zero values of

Δ_s or α_s all scale basis vectors that are no unambiguously defined. This is the key mechanism by which the embedder is able to maintain continuity in its outputs despite having to use the outputs to both encode the coifficients (which change smoothly) and the basis vectors themselves (which change discontinuously at boundaries).

			$\delta_{(s)} - \delta_{(s+1)}$	$L_{(s-1)} + V_{(s)}$	$(s+1)\Delta_{(s)}$	$L_{(s)}/(s+1)$
s	$\delta_{(s)}$	$V_{(s)}$	$\Delta_{(s)}$	$L_{(s)}$	α_s	$\hat{L}_{(s)}$
0	5	$e_0^0 + e_0^2 + e_0^3$	1	$e_0^0 + e_0^2 + e_0^3$	1	$e_0^0 + e_0^2 + e_0^3$
1	4	e_0^2	1	$e_0^0 + 2e_0^2 + e_0^3$	2	$\frac{1}{2}(e_0^0 + 2e_0^2 + e_0^3)$
2	3	$e_1^1 + e_1^2 + e_1^3$	1	$e_0^0 + 2e_0^2 + e_0^3 + e_1^1 + e_1^2 + e_1^3$	3	$\frac{1}{3}(e_0^0 + 2e_0^2 + e_0^3 + e_1^1 + e_1^2 + e_1^3)$
3	2	$e_1^2 + e_1^3$	0	$e_0^0 + 2e_0^2 + e_0^3 + e_1^1 + 2e_1^2 + 2e_1^3$	0	$\frac{1}{4}(e_0^0 + 2e_0^2 + e_0^3 + e_1^1 + 2e_1^2 + 2e_1^3)$
4	2	$e_0^0 + e_0^2$	0	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 2e_1^2 + 2e_1^3$	0	$\frac{1}{5}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 2e_1^2 + 2e_1^3)$
5	2	e_1^2	1	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3$	6	$\frac{1}{6}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3)$
6	1	$e_2^0 + e_2^2 + e_2^3$	0	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + e_2^0 + e_2^2 + e_2^3$	0	$\frac{1}{7}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + e_2^0 + e_2^2 + e_2^3)$
7	1	e_2^0	1	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 2e_2^0 + e_2^2 + e_2^3$	8	$\frac{1}{8}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 2e_2^0 + e_2^2 + e_2^3)$
8	0	$e_2^0 + e_2^2$	0	$2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 3e_2^0 + 2e_2^2 + e_2^3$	0	$\frac{1}{9}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 3e_2^0 + 2e_2^2 + e_2^3)$
	$\sum_s \delta_{(s)} = 20$		$\sum_s \Delta_{(s)} = 5 \neq 20$		$\sum_s \alpha_{(s)} = 20$	
	$\sum_s \delta_{(s)} V_{(s)} =$ $7e_0^0 + 11e_0^2 + 5e_0^3 + 3e_1^1 + 7e_1^2 + 5e_1^3 + 2e_2^0 + e_2^2 + e_2^3$		$\sum_s \Delta_{(s)} L_{(s)} =$ $7e_0^0 + 11e_0^2 + 5e_0^3 + 3e_1^1 + 7e_1^2 + 5e_1^3 + 2e_2^0 + e_2^2 + e_2^3$		$\sum_s \alpha_{(s)} \hat{L}_{(s)} =$ $7e_0^0 + 11e_0^2 + 5e_0^3 + 3e_1^1 + 7e_1^2 + 5e_1^3 + 2e_2^0 + e_2^2 + e_2^3$	

Step E: Canonicalise

Sort the columns of each count matrix lexicographically (the column-sort permutation "forgets" which vector was which, achieving multiset invariance). After canonicalisation, L_s becomes $\widetilde{L_s}$.

Step F: Hash and assemble

Each $\widetilde{L_s}$ (along with its index) is hashed to a point $p_s \in [0,1]^{19}$ (since $N_{\text{hash}} = 2{\times}9{+}1 = 19$). The output is:

$$f(M) \;=\; \big[\, \underbrace{-3,\ 2,\ 1}_{\text{3 row min.}},\ 1\,p_0 + 2\,p_1 + 3\,p_2 + 6\,p_5 + 8\,p_7 \,\big] \;\in\; \mathbb{R}^{22}.$$

5 Worked Example: Algorithm 1, $n = 4$, $k = 2$

$$M \;=\; \left\{ \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \begin{pmatrix} -3 \\ 5 \end{pmatrix}, \begin{pmatrix} 8 \\ 9 \end{pmatrix}, \begin{pmatrix} 2 \\ 7 \end{pmatrix} \right\} \quad (n = 4 \text{ vectors in } \mathbb{R}^2)$$

Step B: Extract global minimum (and maximum)

Global min $= -3$, global max $= 9$. Both stored as anchors. Balance has all non-negative entries.

Steps C–D: Global sort and Abel summation

The balance after subtracting the global minimum -3 from every entry is:

$$\left\{ \binom{7}{5}, \binom{0}{8}, \binom{11}{12}, \binom{5}{10} \right\}$$

Sorted globally descending (value: label):

$$12(e_1^2),\ 11(e_0^2),\ 10(e_1^3),\ 8(e_1^1),\ 7(e_0^0),\ 5(e_1^0),\ 5(e_0^3),\ 0(e_0^1).$$

The 7 gaps are $1, 1, 2, 1, 2, 0, 5$. After global re-sort by gap descending and second Abel summation: $\alpha_0 = 3,\ \alpha_2 = 3,\ \alpha_5 = 6$ (others zero).

$N_{\text{hash}} = 2(8-1)+1 = 15$. Output:

$$f(M) \ = \ \left[\, 3\,p_0 + 3\,p_2 + 6\,p_5,\ 9,\ -3 \,\right] \ \in \ \mathbb{R}^{17},$$

numerically (from the code's debug output):

$$[\,7.07,\ 6.91,\ 2.78,\ 3.50,\ 1.01,\ 5.51,\ 4.07,\ 4.90,\ 10.02,\ 9.51,\ 5.13,\ 2.97,\ 8.04,\ 5.34,\ 7.41,\ 9,\ -3\,].$$

6 Output Size and the Whitney Embedding Theorem

Both algorithms produce output of size $\approx 2nk$, which is not overhead to be engineered away — it is intrinsic. The input space $SP^n(\mathbb{R}^k)$ is an nk-dimensional manifold. The **Whitney Embedding Theorem** states that any smooth d-dimensional manifold embeds smoothly into $\mathbb{R}^{2d}$, and $2d$ is tight in general. With output $\approx 2nk$, both algorithms sit right at the Whitney bound.

The factor of ≈ 2 is in fact the price of the cone-dimension mechanism in Step F: encoding m positive coefficients together with the identities of m basis elements inside a single point requires $\mathbb{R}^{2m+1}$. This same mechanism is what delivers C^0 continuity (shared faces at zero-coefficient transitions). The factor of two is the minimum price of a smooth embedding, and the algorithms pay almost exactly that.

Whether $2nk$ (or $2nk + 1$, or $2nk + 1 - k$) is optimal for the specific manifold $SP^n(\mathbb{R}^k)$, or whether the known structure of that space admits a tighter construction, remains open.

A Source-code Correspondence

Files

File	Role
`C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py`	Algorithm 2 direct; `Embedder.embed_generic`; Steps A–F. Original file on which this document is based.
`C0HomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py`	Algorithm 2 EncDec-backed; `Embedder.embed_generic`; Steps A–F. Delegates Steps A–E to `EncDec.py`; adds Step F directly. Default config is bit-identical to `C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py`.
`C0HomDeg1_simplicialComplex_embedder_2_for_array_of_reals_as_multiset.py`	Wrapper selecting between `C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py` and `C0HomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py`. All other repository code imports `Embedder` from this file.
`C0HomDeg1_simplicialComplex_embedder_1_for_array_of_reals_as_multiset.py`	Algorithm 1 direct; `Embedder.embed_generic`; Steps A–F
`EncDec.py`	`simplex_2_preprocess_steps` and `simplex_1_preprocess_steps`; Steps A–E only
`play_simplex_encoders_via_EncDec.py`	Command-line driver for `EncDec.py`

Notation–code correspondence (`EncDec.py`)

The central function is `barycentric_subdivide(lin_comb, ...)`, which performs one Abel-summation step. Both algorithms call it twice: once to extract the symmetric anchor(s) and once to produce the final coefficient–basis pairs. The table below maps the document's symbols to the corresponding names in `EncDec.py`.

Document symbol	`EncDec.py` name / expression	Notes
Input M ($k\times n$)	`set_array`	Rows = coordinates i; columns = vectors j
e_i^j (Eji basis matrix)	`basis_vec` inside `MonoLinComb`	$k\times n$ indicator: 1 at (i,j), 0 elsewhere
Step A: $M = \sum_{j,i} X[j][i]\, e_i^j$	`array_to_lin_comb(set_array)` → `LinComb`	Each `(coeff, basis_vec)` term is one Eji
Anchors m_i (Alg 2); global min (Alg 1)	`offset` returned by `barycentric_subdivide(..., return_offset_separately=True)`	k per-row offsets (Alg 2); one global offset (Alg 1)
$\delta_{(s)}$, $V_{(s)}$	`.coeffs[s]`, `.basis_vecs[s]` of `lin_comb_1_first_diffs`	After 1st `barycentric_subdivide`; sorted descending
$\Delta_{(s)}$, $L_{(s)}$	`.coeffs[s]`, `.basis_vecs[s]` of `lin_comb_2_second_diffs`	2nd call with `preserve_scale=False`
α_s, $\hat{L}_{(s)}$	same	2nd call with `preserve_scale=True` (default); `coeffs[s]` $= (s{+}1)\Delta_{(s)}$, `basis_vecs[s]` $= L_{(s)}/(s{+}1)$
$\widetilde{L}_{(s)}$ (canonical)	`tools.sort_np_array_rows_lexicographically(basis_vec)`	Step E; sorts columns of each basis matrix
Step F hash	`Eji_LinComb.hash_to_point_in_unit_hypercube`	In `Eji_LinComb.py`; not yet in `EncDec.py`

The `preserve_scale` flag controls normalisation of prefix sums: `True` gives $\hat{L}_{(s)} = L_{(s)}/(s{+}1)$ with $\alpha_s = (s{+}1)\Delta_{(s)}$; `False` gives raw $L_{(s)}$ with $\Delta_{(s)}$. The second `barycentric_subdivide` call uses `preserve_scale=True` by default, matching the $\alpha_s/\hat{L}_{(s)}$ notation used throughout this document.

Differences between the two implementations

- **Step F coverage.** `EncDec.py` covers only Steps A–E. The `COHomDeg1` files complete the embedding by hashing each canonical basis matrix to a point in the unit hypercube via `Eji_LinComb.hash_to_point_in_unit_hypercube` and forming $\sum_s \alpha_s p_s$. In `COHomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py`, Step F is implemented directly in `_hash_basis_vec_to_point`, which reconstructs the integer count matrix from `EncDec.py`'s Fraction-valued basis vectors and then applies the same MD5→seed→RNG hash as `Eji_LinComb`.
- **Arithmetic.** `EncDec.py` uses `fractions.Fraction` for coefficients when `preserve_scale=True`, giving exact rational arithmetic. The `COHomDeg1` files use `numpy` integers throughout.
- **Internal representation.** The `COHomDeg1` files use the `Eji`, `Eji_LinComb`, and `Maximal_Simplex_Vertex` classes from their own modules. `EncDec.py` uses plain `numpy` arrays as basis vectors inside `MonoLinComb`/`LinComb`.